Guía de Actividades **Práctico-Experimentales** Nro. 005

1. Datos Generales

Nombres de los Estudiantes	Christopher Alexander Pineda Rodas Eduardo José Soto Cabrera Lenin Fabricio Macas Cabrera Willan Sebastian Granda Camacho Jeancarlos Fernando Aguirre Romero
Asignatura	Estructura de Datos
Ciclo	3ro A
Unidad	1: Estructuras de datos estáticas y dinámicas
Resultado de aprendizaje de la unidad	Listas Doblemente Enlazadas y Pilas: Navegación Bidireccional y Gestión LIFO de Mínimo Costo Energético
Práctica Nro.	005
Título de la Práctica	Listas Doblemente Enlazadas y Pilas: Navegación Bidireccional y Gestión LIFO de Mínimo Costo Energético
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 22 y viernes 23 de octubre 2026
Horario	Jueves 29 de octubre 10:30 – 11:30 Viernes 30 de octubre 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

- Modear e implementar en **JavaScript (Node.js)** una estructura de **Lista Doblemente Enlazada** y una **Pila (Stack LIFO)** para la simulación de un módulo de navegación bidireccional y un sistema transaccional de deshacer/rehacer (*Undo/Redo*) para el proyecto de gestión satelital sustentable.
- Evaluar experimentalmente la eficiencia temporal de inserción/eliminación en extremos frente a la reestructuración por desplazamiento en arreglos nativos.
- Cuantificar el consumo de memoria Heap generado por el almacenamiento de punteros dobles y mediante métricas del motor V8, garantizando el cumplimiento de criterios de Sostenibilidad Computacional (*Green Computing*).

3. Materiales y reactivos:

- Entorno de ejecución Node.js (versión LTS 20.x o superior) instalado.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación de estructuras dinámicas.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** Los estudiantes resolverán el reto de diseñar un subsistema de telemetría y auditoría de un satélite sustentable, el cual requiere almacenar eventos en un historial de navegación bidireccional (adelante/atrás) y ejecutar comandos de reversión inmediata de fallos en modo LIFO (*Last In, First Out*).

- **ABI (Aprendizaje Basado en Investigación):** Investigación empírica sobre la huella espacial de los punteros de 64 bits en la V8 Engine y la justificación matemática de la compensación entre espacio y tiempo (*Space-Time Trade-off*).

Tarea 1: Construcción de la Lista Doblemente Enlazada (Navegación Bidireccional)

1. En la carpeta del proyecto /APE_005_Listas_Pilas, cree el archivo ListaDoble.js.
2. Implemente la clase NodoDoble y la clase ListaDoblementeEnlazada garantizando que las referencias cruzadas se actualicen en tiempo constante O1.

JavaScript

```
/**
 * Representa un Nodo en una Lista Doblemente Enlazada.
 */
class NodoDoble {
  constructor(datos) {
    this.datos = datos; // Carga útil de telemetría satelital
    this.next = null;   // Referencia al siguiente nodo
    this.prev = null;   // Referencia al nodo anterior
  }
}

/**
 * Estructura de Datos Dinámica Doblemente Enlazada.
 */
class ListaDoblementeEnlazada {
  constructor() {
    this.head = null;
    this.tail = null;
    this.longitud = 0;
  }

  /**
   * Inserta un nuevo elemento al final de la lista.
   * Complejidad Temporal: O(1)
   */
  insertarFinal(datos) {
    const nuevoNodo = new NodoDoble(datos);
    if (!this.head) {
      this.head = nuevoNodo;
      this.tail = nuevoNodo;
    } else {
      this.tail.next = nuevoNodo;
      nuevoNodo.prev = this.tail;
      this.tail = nuevoNodo;
    }
    this.longitud++;
    return nuevoNodo;
```

```
  }

  /**
   * Elimina el último elemento de la lista.
   * Complejidad Temporal: O(1)
   */
  eliminarFinal() {
    if (!this.tail) return null;
    const nodoEliminado = this.tail;
    if (this.head === this.tail) {
      this.head = null;
      this.tail = null;
    } else {
      this.tail = this.tail.prev;
      this.tail.next = null;
    }
    nodoEliminado.prev = null;
    this.longitud--;
    return nodoEliminado.datos;
  }
}

module.exports = { NodoDoble, ListaDoblementeEnlazada };
```

Tarea 2: Implementación de la Pila Transaccional (Stack LIFO) sobre Punteros

1. Cree el archivo PilaTransaccional.js.
2. Desarrolle la Pila de comandos satelitales apoyada en nodos enlazados para evitar la relocalización de memoria de los arreglos dinámicos al hacer push o pop.

JavaScript

```
const { ListaDoblementeEnlazada } = require('./ListaDoble');

/**
 * Pila (Stack) orientada a Sostenibilidad Computacional.
 * Operaciones fundamentales Push y Pop en O(1) estricto.
 */
class PilaTransaccional {
  constructor() {
    this.lista = new ListaDoblementeEnlazada();
  }

  /**
   * Apila un comando de telemetría en el tope.
   * Complejidad Temporal: O(1)
   */
  push(comando) {
    this.lista.insertarFinal(comando);
  }
```

```
 /**
  * Desapila y retorna el comando en el tope (Undo).
  * Complejidad Temporal: O(1)
  */
 pop() {
  return this.lista.eliminarFinal();
 }

 /**
  * Retorna la cantidad de elementos almacenados.
  */
 obtenerTamano() {
  return this.lista.longitud;
 }
}

module.exports = PilaTransaccional;
```

Tarea 3: Experimento Comparativo (Benchmark CPU & Memory Heap)

1. Cree el archivo BenchmarkNavegacion.js.
2. Implemente un script que ejecute $N = 500.000$ inserciones y extracciones al inicio/final comparando un Array nativo de JavaScript frente a la PilaTransaccional.

JavaScript

```
const PilaTransaccional = require('./PilaTransaccional');

function obtenerMemoriaMB() {
 return (process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2);
}

const N = 500000;
console.log(`=== BENCHMARK DE RENDIMIENTO (N = ${N}) ===\n`);

// 1. EVALUACIÓN CON ARREGLO TRADICIONAL (unshift / shift)
if (global.gc) global.gc();
const memInicialArray = process.memoryUsage().heapUsed;
console.time('Tiempo Array Nativo (unshift/shift)');

const arregloNativo = [];
for (let i = 0; i < N; i++) {
 arregloNativo.unshift({ id: i, log: 'Evento Satelital' }); // O(n)
}
for (let i = 0; i < N; i++) {
 arregloNativo.shift(); // O(n)
}

console.timeEnd('Tiempo Array Nativo (unshift/shift)');
```

```
const memFinalArray = process.memoryUsage().heapUsed;
const deltaMemArray = ((memFinalArray - memInicialArray) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Array: ${deltaMemArray} MB\n`);

// 2. EVALUACIÓN CON PILA DE PUNTEROS DINÁMICOS
if (global.gc) global.gc();
const memInicialPila = process.memoryUsage().heapUsed;
console.time('Tiempo Pila Transaccional O(1)');

const pilaSatelital = new PilaTransaccional();
for (let i = 0; i < N; i++) {
  pilaSatelital.push({ id: i, log: 'Evento Satelital' }); // O(1)
}
for (let i = 0; i < N; i++) {
  pilaSatelital.pop(); // O(1)
}

console.timeEnd('Tiempo Pila Transaccional O(1)');
const memFinalPila = process.memoryUsage().heapUsed;
const deltaMemPila = ((memFinalPila - memInicialPila) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Pila: ${deltaMemPila} MB\n`);
```

3. Ejecute el script con la bandera de recolección de basura:

   Bash

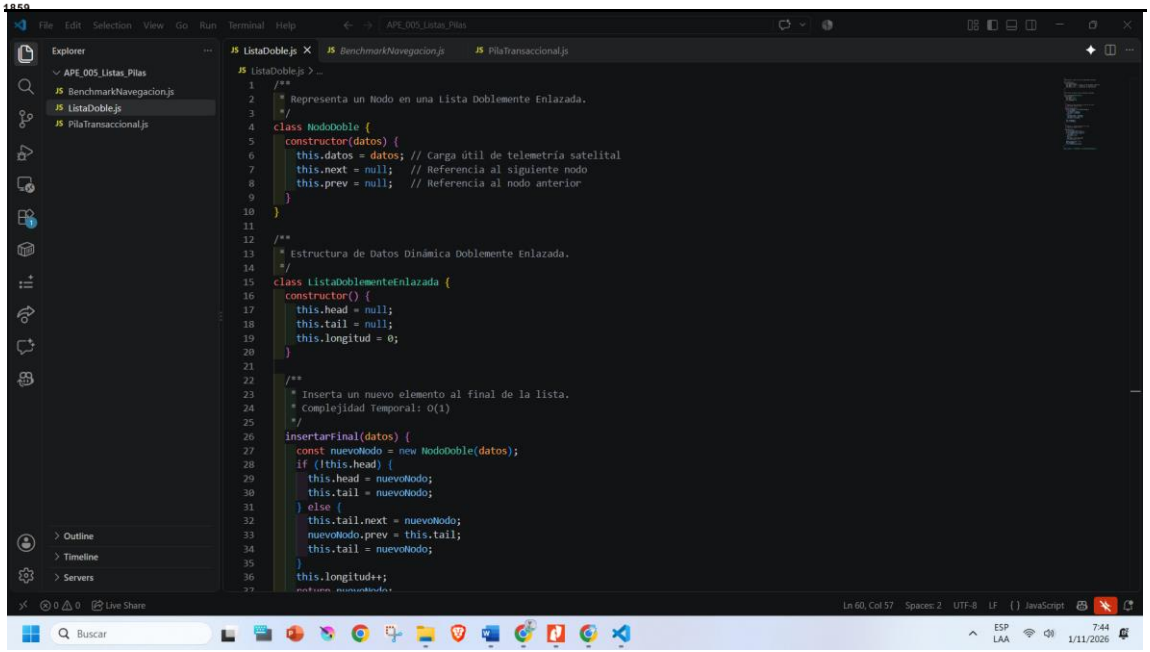
   ```
   node --expose-gc BenchmarkNavegacion.js
   ```

6. Resultados Esperados

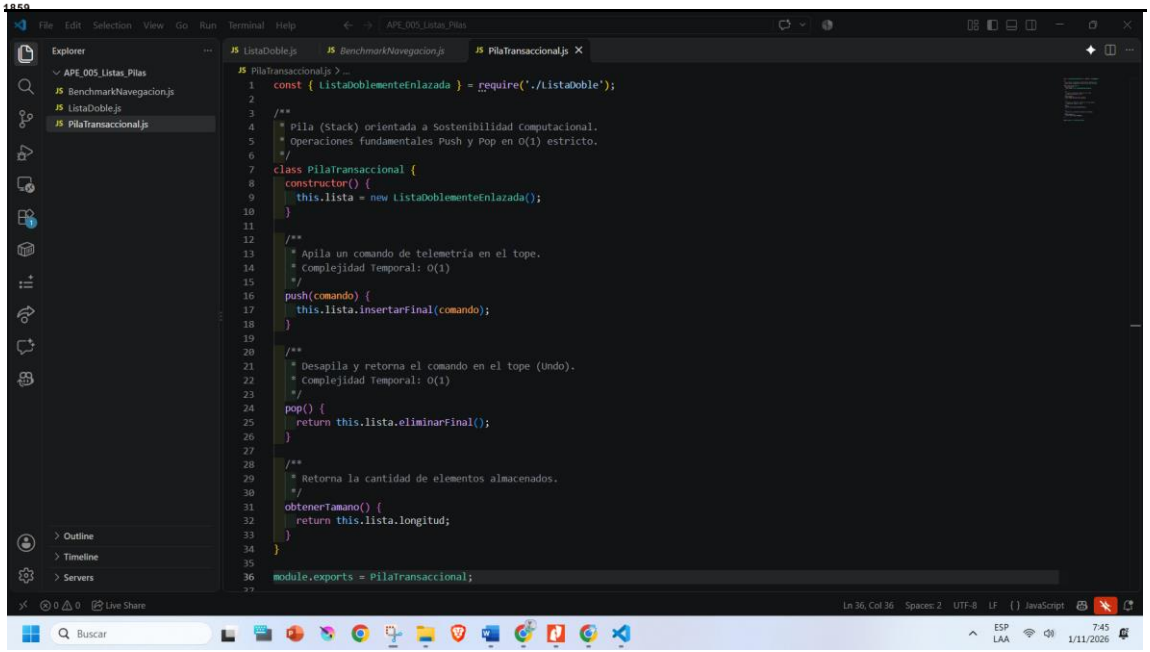
El estudiante evidenciará a través Registro de ejecución en la consola donde se evidencia que las operaciones de inserción y extracción masiva mediante la PilaTransaccional reducen drásticamente el tiempo de ejecución en milisegundos ms en comparación con unshift() y shift() del arreglo nativo, demostrando la diferencia matemática.

- Creación de los códigos de la actividad:

  - ListaDoble.js



- PilaTransaccional.js



```
const { ListaDoblementeEnlazada } = require('./ListaDoble');

/**
 * Pila (Stack) orientada a Sostenibilidad Computacional.
 * Operaciones fundamentales Push y Pop en O(1) estricto.
 */
class PilaTransaccional {
  constructor() {
    this.lista = new ListaDoblementeEnlazada();
  }

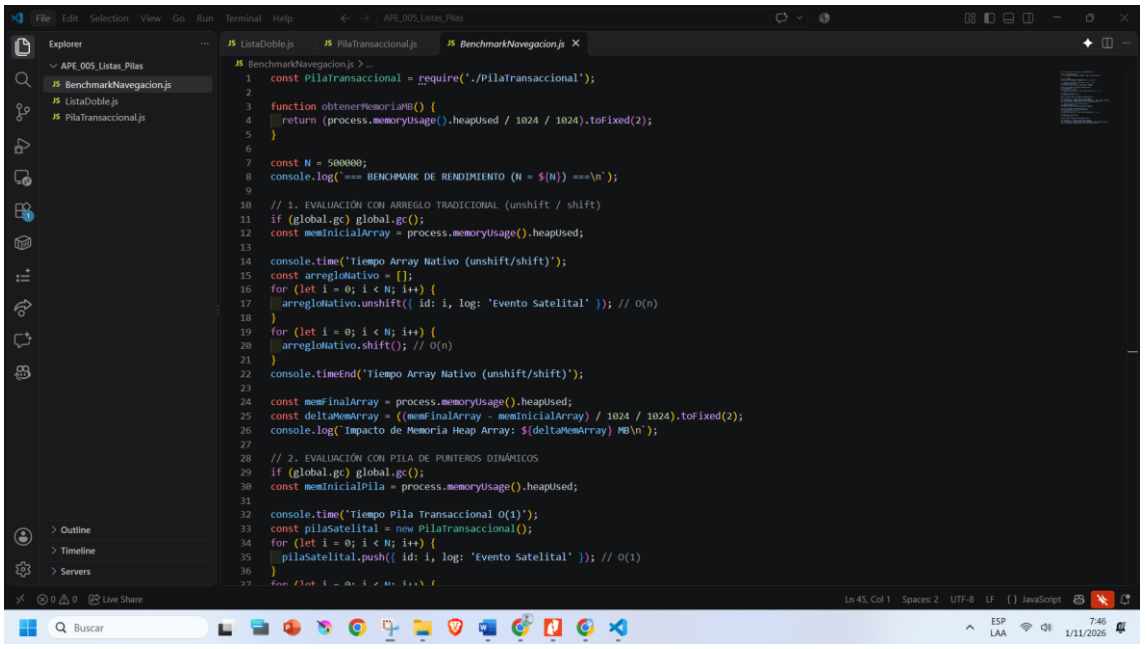
  /**
   * Apila un comando de telemetría en el tope.
   * Complejidad Temporal: O(1)
   */
  push(comando) {
    this.lista.insertarFinal(comando);
  }

  /**
   * Desapila y retorna el comando en el tope (Undo).
   * Complejidad Temporal: O(1)
   */
  pop() {
    return this.lista.eliminarFinal();
  }

  /**
   * Retorna la cantidad de elementos almacenados.
   */
  obtenerTamano() {
    return this.lista.longitud;
  }
}

module.exports = PilaTransaccional;
```

- BenchmarkNavegacion.js



```
const PilaTransaccional = require('./PilaTransaccional');

function obtenerMemoriaMB() {
  return (process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2);
}

const N = 500000;
console.log(`=== BENCHMARK DE RENDIMIENTO (N = ${N}) ===\n`);

// 1. EVALUACIÓN CON ARREGLO TRADICIONAL (unshift / shift)
if (global.gc) global.gc();
const memInicialArray = process.memoryUsage().heapUsed;

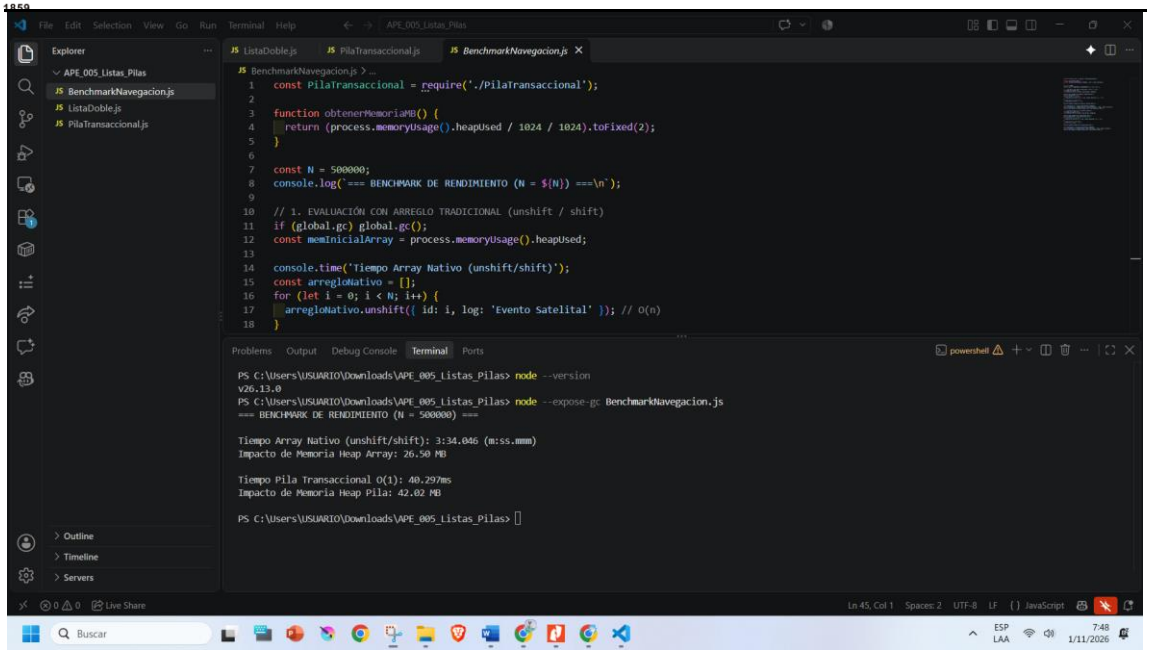
console.time('Tiempo Array Nativo (unshift/shift)');
const arregloNativo = [];
for (let i = 0; i < N; i++) {
  arregloNativo.unshift({ id: i, log: 'Evento Satelital' }); // O(n)
}
for (let i = 0; i < N; i++) {
  arregloNativo.shift(); // O(n)
}
console.timeEnd('Tiempo Array Nativo (unshift/shift)');

const memFinalArray = process.memoryUsage().heapUsed;
const deltaMemArray = ((memFinalArray - memInicialArray) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Array: ${deltaMemArray} MB\n`);

// 2. EVALUACIÓN CON PILA DE PUNTEROS DINÁMICOS
if (global.gc) global.gc();
const memInicialPila = process.memoryUsage().heapUsed;

console.time('Tiempo Pila Transaccional O(1)');
const pilaSatelital = new PilaTransaccional();
for (let i = 0; i < N; i++) {
  pilaSatelital.push({ id: i, log: 'Evento Satelital' }); // O(1)
}
```

- Ejecución de “node --expose-gc” BenchmarkNavegacion.js



- Tabla comparativa de los resultados de ejecucion

Estructura	Operaciones	Tiempo	Heap (MB)
Array nativo (`unshift/shift`)	500.000 + 500.000	3:34.046 (≈ 214.046 ms)	26,50
Pila Transaccional (lista doble)	500.000 + 500.000	40,297 ms	42,02

- Explicación de los resultados de la ejecución

Con N = 500.000, la Pila Transaccional completó las inserciones y extracciones en 40,297 ms, mientras que el arreglo nativo con unshift()/shift() tardó 3 min 34,046 s (≈ 214.046 ms). La pila resultó aproximadamente 5.310 veces más rápida. Esto se explica porque unshift() y shift() desplazan todos los elementos del arreglo en cada operación (O(n) por operación, $O(n^2)$ en total), mientras que push() y pop() de la pila solo reasignan los punteros next y prev (O(1) por operación, O(n) en total)

En memoria Heap, la pila registró 42,02 MB frente a 26,50 MB del arreglo, unos 15,5 MB más. Esa diferencia se debe a que cada nodo almacena dos punteros adicionales (next y prev) además del dato, lo que ilustra el compromiso espacio-tiempo (Space-Time Trade-off): la pila gasta más memoria, pero reduce drásticamente el tiempo de ejecución, y menos tiempo de CPU significa menos consumo energético.

7. Preguntas de Control:

Responda fundamentando sus argumentos bajo una perspectiva lógica, algorítmica y filosófica:

- **En el contexto de las estructuras de datos dinámicas, ¿cuál es la "sustancia" de un nodo en una Lista Doblemente Enlazada en comparación con la de una celda de un arreglo contiguo, y cómo modifica el puntero anterior (prev) su naturaleza constitutiva?**

La celda de un arreglo no tiene identidad propia: es una posición numerada dentro de un bloque continuo, y su sentido depende de su índice. Por eso insertar o eliminar al inicio obliga a desplazar todas las demás celdas. El nodo, en cambio, es una entidad autónoma que encapsula su dato junto con las referencias a sus vecinos. Su sustancia no está en el lugar físico que ocupa, sino en las relaciones que mantiene, y puede estar en cualquier dirección del Heap sin perder su lugar en la secuencia. El puntero prev lo vuelve bidireccional: el nodo deja de conocer solo su futuro (next) y adquiere memoria de su origen. Así puede retroceder, y eliminar el último elemento sin recorrer la lista, algo que con un solo puntero costaría O(n).

- **¿Por qué la operación de inserción y extracción LIFO en una Pila implementada mediante punteros dobles alcanza la causa eficiente de la inmediatez O1, superando el trabajo mecánico de reubicación física de celdas que realiza el motor V8 en un arreglo nativo?**

En la pila, push y pop solo tocan el extremo (tail): crear el nodo, enlazar tail.next y nuevoNodo.prev, y mover tail. Son un número fijo de asignaciones, independiente de cuántos elementos existan. La causa eficiente de esa inmediatez es que el orden lógico está en los punteros y no en la posición física, así que ningún otro elemento necesita enterarse de la operación. En un arreglo nativo, unshift y shift cambian la posición de todos los elementos, por lo que el motor debe copiar o desplazar hasta n celdas en cada llamada (O(n) por operación, $O(n^2)$ en total para N operaciones). Lo confirma el benchmark: con N = 500.000 la pila tardó 40,297 ms y el arreglo 3 min 34 s.

- **¿Cuál es el fin último (*telos*) de la navegación bidireccional en un historial de operaciones o sistema de transacciones *Undo/Redo*, y en qué medida optimiza la estabilidad energética del sistema de cómputo?**

El fin último de la navegación bidireccional es la reversibilidad: que el sistema pueda deshacer un estado erróneo y rehacerlo sin reconstruirlo desde cero. En el satélite sustentable, eso significa corregir un comando fallido al instante, sin recalcular ni reprocesar todo el historial. Para esto la lista doble guarda el pasado y el futuro de cada evento enlazados, y moverse entre ellos cuesta O(1). Energéticamente, evitar reconstrucciones y desplazamientos masivos reduce los ciclos de CPU, y por tanto el consumo y el calor. Esa mejora es proporcional al tiempo ahorrado, que en el benchmark es enorme. Pero es una mejora indirecta: el beneficio viene del algoritmo, no de que la estructura sea “más verde” en sí misma.

- **Aristóteles sostenía que nada existe sin una forma y materia dada. Si para ganar flexibilidad bidireccional se requiere duplicar las referencias de memoria prev y next, ¿cuándo deja el gasto de memoria Heap de ser una virtud de diseño para convertirse en un exceso ontológico (*overhead*)?**

La forma (los punteros next y prev) es lo que permite la bidireccionalidad, así que mientras esa capacidad se use, el gasto está justificado. En nuestro experimento la pila usó 42,02 MB frente a 26,50 MB del arreglo, unos 15,5 MB más, a cambio de reducir el tiempo de miles de veces. Deja de ser virtud y se convierte en exceso cuando la forma ya no cumple su función. Por ejemplo, cuando solo se opera en un extremo y nunca se usa prev para retroceder, cuando los datos son tan pequeños que el puntero pesa más que la carga útil, o cuando la memoria es el recurso más escaso del sistema (un satélite con RAM limitada). En esos casos conviene una estructura más simple, como una lista simple o un arreglo.

- **Considerando la conservación de energía como principio técnico y ético, ¿de qué manera la eliminación instantánea de referencias sin desplazamiento de elementos mitiga el trabajo electromagnético de la CPU?**

Eliminar un nodo solo cambia un par de referencias, mientras que desplazar elementos obliga a leer y escribir miles o millones de celdas, moviendo datos entre memoria, caché y registros. Cada una de esas operaciones consume ciclos de CPU, y el consumo energético crece con la cantidad de ciclos y accesos a memoria que el procesador ejecuta. Al evitar el desplazamiento, la CPU realiza mucho menos trabajo para obtener el mismo resultado, y eso se traduce en menos energía y menos calor. Desde lo ético, es un uso mesurado del recurso: no gastar energía en mover lo que no necesita moverse. Como matiz honesto, las listas enlazadas tienen peor localidad de caché que los arreglos y más presión sobre el recolector de basura, así que la ventaja energética es clara en este caso (operaciones en los extremos) pero no es universal.

8. Evaluación

Criterio Evaluado	Evidencia Esperada	Puntaje Máximo
Análisis Green Computing	Justificación técnica del ahorro de CPU vs gasto de RAM.	2.0
Arquitectura de Nodos	Implementación correcta de la referencia bidireccional.	4.0
Navegación Fluida	Métodos next() y prev() funcionales con complejidad .	2.0
Defensa Técnica	Explicación video del impacto de los punteros en la energía.	2.0
TOTAL		**10.0**

9. Bibliografía

Liste las fuentes utilizadas. Use normas **IEEE**. Pueden ser libros, manuales, artículos, tutoriales académicos, documentación oficial de software.

[1] B. Goetz, Java Concurrency in Practice. Addison-Wesley, 2022.

[2] P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023.

[3] J. Bloch, Effective Java, 3rd ed. Boston, MA: Addison-Wesley, 2018.

10. Elaboración y Aprobación

- Elaborado por: nombre del Docente responsable de la práctica.
- Aprobado por: nombre del Director de Carrera.

Elaborado por	Cristian R. Narváez Guillén **Docente**	
Aprobado por	Edison L Coronel Romero **Director de Carrera**	